
Data Structures

BS (CS) _Fall_2025

Lab_07 Task



Learning Objectives:

1. Sorting using Arrays and Linked Lists

Task 1:

Problem Statement:

Write a C++ program to implement Quick Sort on a singly linked list.

Requirements:

Define a Node class with data and next members.

Implement the following functions:

- **Node* getTail(Node* head)** → Finds the last node of the linked list.
- **Node* partition(Node* head, Node* tail)** → Partitions the linked list around a pivot.
- **void quickSortHelper(Node* head, Node* tail)** → Recursively applies Quick Sort on sublists.
- **Node* quickSort(Node* head)** → Wrapper function that calls quickSortHelper.
- **void printList(Node* head)** → Displays the linked list in proper format.

Complexity Analysis:

- Write a function that displays the size of list and show the number of iterations it takes for your Quick Sort algorithm to sort a given list.
- Perform Complexity Analysis on your code. Show your work on a piece of paper.

Algorithm

QuickSort Function

- Find the last node of the linked list using the GetTail function.
- Call the QuickSortHelper function with the head node and the last node.
- Return the head of the sorted list.

QuickSortHelper Function

- If the list is empty or contains only one node, stop (base case).
- Choose a pivot by calling the Partition function.
- Recursively sort the portion of the list that comes before the pivot.
- Recursively sort the portion of the list that comes after the pivot.

Partition Function

- Select the first node of the list as the pivot.
- Use two pointers to track the current node being checked and the boundary of elements smaller than the pivot.
- Traverse the list from the head to the tail:
 - For each node with a value smaller than the pivot, move it just after the boundary pointer and update the boundary.

- After traversal, place the pivot in its correct position by swapping it with the boundary node.
- Return the boundary node as the new pivot position.

GetTail Function

- Starting from a given node, move forward through the linked list until the last node is reached.
- Return this last node.

Expected Output:

Original List: 30 -> 3 -> 4 -> 20 -> 5

Sorted List: 3 -> 4 -> 5 -> 20 -> 30

Test Cases:

```
#include <gtest/gtest.h>

// Test cases for Quick Sort on Singly Linked List

// Test empty list
TEST(QuickSortTest, EmptyListInt) {
    LinkedList<int> list;
    list.quickSort();
    // Empty list should remain empty after sorting
    EXPECT_TRUE(list.isEmpty());
}

// Test single element list
TEST(QuickSortTest, SingleElementInt) {
    LinkedList<int> list;
    list.insertAtHead(42);
    list.quickSort();

    int expected[] = {42};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 1; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}
```

```

// Test two elements - already sorted
TEST(QuickSortTest, TwoElementsSortedInt) {
    LinkedList<int> list;
    list.insertAtHead(2);
    list.insertAtHead(1);
    list.quickSort();

    int expected[] = {1, 2};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 2; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test two elements - reverse order
TEST(QuickSortTest, TwoElementsReverseInt) {
    LinkedList<int> list;
    list.insertAtHead(3);
    list.insertAtHead(5);
    list.quickSort();

    int expected[] = {3, 5};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 2; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test three elements - various orders
TEST(QuickSortTest, ThreeElementsInt) {
    LinkedList<int> list;
    list.insertAtHead(1);
    list.insertAtHead(3);
    list.insertAtHead(2);
    list.quickSort();

    int expected[] = {1, 2, 3};
    Node<int>* current = list.getHead();

```

```

    for (int i = 0; i < 3; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test odd number of elements
TEST(QuickSortTest, OddElementsInt) {
    LinkedList<int> list;
    int input[] = {64, 34, 25, 12, 22};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {12, 22, 25, 34, 64};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test even number of elements
TEST(QuickSortTest, EvenElementsInt) {
    LinkedList<int> list;
    int input[] = {38, 27, 43, 3, 9, 82};
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {3, 9, 27, 38, 43, 82};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

```

```

// Test already sorted list
TEST(QuickSortTest, AlreadySortedInt) {
    LinkedList<int> list;
    int input[] = {7, 6, 5, 4, 3, 2, 1}; // Insert in reverse to get sorted order
    for (int i = 0; i < 7; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {1, 2, 3, 4, 5, 6, 7};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 7; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test reverse sorted list
TEST(QuickSortTest, ReverseSortedInt) {
    LinkedList<int> list;
    int input[] = {5, 6, 7, 8, 9, 10}; // Insert to get reverse order
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {5, 6, 7, 8, 9, 10};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test with duplicate elements
TEST(QuickSortTest, DuplicateElementsInt) {
    LinkedList<int> list;
    int input[] = {5, 1, 9, 2, 8, 2, 5, 5};
    for (int i = 0; i < 8; i++) {
        list.insertAtHead(input[i]);
    }

```

```

    }
    list.quickSort();

    int expected[] = {1, 2, 2, 5, 5, 5, 8, 9};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 8; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test with all same elements
TEST(QuickSortTest, AllSameElementsInt) {
    LinkedList<int> list;
    int input[] = {7, 7, 7, 7, 7};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {7, 7, 7, 7, 7};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test with negative numbers
TEST(QuickSortTest, NegativeNumbersInt) {
    LinkedList<int> list;
    int input[] = {2, 0, -1, 7, -5, -3};
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {-5, -3, -1, 0, 2, 7};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);

```

```

        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test large list
TEST(QuickSortTest, LargeListInt) {
    LinkedList<int> list;
    int input[] = {125, 300, 25, 50, 75, 150, 2, 3, 1, 4, 200, 100};
    for (int i = 0; i < 12; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {1, 2, 3, 4, 25, 50, 75, 100, 125, 150, 200, 300};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 12; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test worst case scenario (already sorted - bad for basic quicksort)
TEST(QuickSortTest, WorstCaseInt) {
    LinkedList<int> list;
    int input[] = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1}; // Insert to create sorted
list
    for (int i = 0; i < 10; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 10; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

```



```

// Test using insertAtTail
TEST(QuickSortTest, InsertAtTailInt) {
    LinkedList<int> list;
    int input[] = {64, 34, 25, 12, 22};
    for (int i = 0; i < 5; i++) {
        list.insertAtTail(input[i]);
    }
    list.quickSort();

    int expected[] = {12, 22, 25, 34, 64};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test with random pattern
TEST(QuickSortTest, RandomPatternInt) {
    LinkedList<int> list;
    int input[] = {45, 12, 78, 34, 67, 23, 89, 56, 90, 11};
    for (int i = 0; i < 10; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    int expected[] = {11, 12, 23, 34, 45, 56, 67, 78, 89, 90};
    Node<int>* current = list.getHead();
    for (int i = 0; i < 10; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Float data type tests
TEST(QuickSortTest, FloatElements) {
    LinkedList<float> list;
    float input[] = {0.9f, 2.1f, 4.8f, 1.2f, 3.5f};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
}

```

```

list.quickSort();

float expected[] = {0.9f, 1.2f, 2.1f, 3.5f, 4.8f};
Node<float>* current = list.getHead();
for (int i = 0; i < 5; i++) {
    EXPECT_NE(current, nullptr);
    EXPECT_FLOAT_EQ(current->data, expected[i]);
    current = current->next;
}
EXPECT_EQ(current, nullptr);
}

// Float with negative values
TEST(QuickSortTest, FloatNegativeElements) {
    LinkedList<float> list;
    float input[] = {4.2f, 0.0f, -1.1f, 3.7f, -2.5f};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    float expected[] = {-2.5f, -1.1f, 0.0f, 3.7f, 4.2f};
    Node<float>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_FLOAT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Float with duplicates
TEST(QuickSortTest, FloatDuplicates) {
    LinkedList<float> list;
    float input[] = {2.5f, 1.5f, 2.5f, 3.0f, 1.5f, 4.0f};
    for (int i = 0; i < 6; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    float expected[] = {1.5f, 1.5f, 2.5f, 2.5f, 3.0f, 4.0f};
    Node<float>* current = list.getHead();
    for (int i = 0; i < 6; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_FLOAT_EQ(current->data, expected[i]);
    }
}

```

```

        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Double data type tests
TEST(QuickSortTest, DoubleElements) {
    LinkedList<double> list;
    double input[] = {2.718, 3.141, 5.555, 1.234, 9.876};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    double expected[] = {1.234, 2.718, 3.141, 5.555, 9.876};
    Node<double>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_DOUBLE_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Double with scientific notation
TEST(QuickSortTest, DoubleScientificNotation) {
    LinkedList<double> list;
    double input[] = {1.1e-1, 8.9e0, 4.7e1, 2.3e-2, 1.5e3};
    for (int i = 0; i < 5; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    double expected[] = {2.3e-2, 1.1e-1, 8.9e0, 4.7e1, 1.5e3};
    Node<double>* current = list.getHead();
    for (int i = 0; i < 5; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_DOUBLE_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Double with high precision
TEST(QuickSortTest, DoublePrecision) {

```

```

    LinkedList<double> list;
    double input[] = {3.141592653589793, 2.718281828459045, 1.414213562373095,
1.732050807568877};
    for (int i = 0; i < 4; i++) {
        list.insertAtHead(input[i]);
    }
    list.quickSort();

    double expected[] = {1.414213562373095, 1.732050807568877, 2.718281828459045,
3.141592653589793};
    Node<double>* current = list.getHead();
    for (int i = 0; i < 4; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_DOUBLE_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test single element with different data types
TEST(QuickSortTest, SingleElementFloat) {
    LinkedList<float> list;
    list.insertAtHead(42.5f);
    list.quickSort();

    float expected[] = {42.5f};
    Node<float>* current = list.getHead();
    for (int i = 0; i < 1; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_FLOAT_EQ(current->data, expected[i]);
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

TEST(QuickSortTest, SingleElementDouble) {
    LinkedList<double> list;
    list.insertAtHead(42.123456789);
    list.quickSort();

    double expected[] = {42.123456789};
    Node<double>* current = list.getHead();
    for (int i = 0; i < 1; i++) {
        EXPECT_NE(current, nullptr);
        EXPECT_DOUBLE_EQ(current->data, expected[i]);
    }
}

```

```
        current = current->next;
    }
    EXPECT_EQ(current, nullptr);
}

// Test empty lists for different data types
TEST(QuickSortTest, EmptyListFloat) {
    LinkedList<float> list;
    list.quickSort();
    EXPECT_TRUE(list.isEmpty());
}

TEST(QuickSortTest, EmptyListDouble) {
    LinkedList<double> list;
    list.quickSort();
    EXPECT_TRUE(list.isEmpty());
}
```